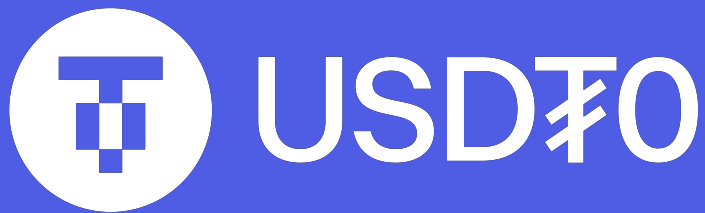


Stellar OFT Audit



April 24, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Contract Architecture	6
Cross-Chain Message Flow	6
Decimal Conversion	7
Security Model and Trust Assumptions	7
Privileged Roles	8
Low Severity	10
L-01 quote_off Can Return An Incorrect max_amount_Id	10
L-02 Fee Configuration Can Create an Unusable State When No Fee Deposit Address Is Set	11
L-03 Pausing __credit Can Strand In-flight Cross-chain Transfers	12
L-04 Blacklisted Recipients Can Cause Stuck Cross-Chain Messages	13
Notes & Additional Information	14
N-01 LockUnlock Mode Lacks End-to-End Test Coverage	14
N-02 Unused SACAdminWrapperClient	15
N-03 unwrap() Breaks Error Pattern	15
Conclusion	16
Appendix	17
Issue Classification	17

Summary

Type	Token	Total Issues	7 (2 resolved)
Timeline	From 2026-03-23 To 2026-04-01	Critical Severity Issues	0 (0 resolved)
Languages	Rust	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	4 (1 resolved)
		Notes & Additional Information	3 (1 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the [Everdawn-Labs/usdt0-oft-contracts](https://github.com/Everdawn-Labs/usdt0-oft-contracts) repository at the [ef908d11db6a0552b3c21bcdea44bdd7a43d580a](https://github.com/Everdawn-Labs/usdt0-oft-contracts/commit/ef908d11db6a0552b3c21bcdea44bdd7a43d580a) commit.

In scope were the following files:

```
stellar-contracts
├── oft/src
│   ├── lib.rs
│   ├── oft.rs
│   ├── extensions
│   │   ├── mod.rs
│   │   ├── oft_fee.rs
│   │   ├── pausable.rs
│   │   └── rate_limiter.rs
│   ├── interfaces
│   │   ├── mod.rs
│   │   └── mintable.rs
│   └── oft_types
│       ├── mod.rs
│       ├── lock_unlock.rs
│       └── mint_burn.rs
└── sac-manager/src
    ├── lib.rs
    ├── sac_manager.rs
    ├── storage.rs
    └── interfaces
        ├── mod.rs
        └── sac_admin_wrapper.rs
```

System Overview

The audited scope covers the **Stellar/Soroban implementation of the USDT0 Omnichain Fungible Token (OFT)**, built on top of the LayerZero V2 messaging protocol. The system enables cross-chain USDT0 transfers between Stellar and other LayerZero-connected blockchains. It consists of two Soroban smart contracts, the **OFT contract** and the **SAC Manager contract**, along with several extension modules that provide fee collection, pausability, and rate limiting.

The OFT contract is an OApp (Omnichain Application) that integrates with a LayerZero V2 Endpoint deployed on Stellar. When a user initiates a cross-chain transfer, the OFT debits tokens locally (via burn or lock), constructs a LayerZero message, and sends it to the Endpoint for cross-chain relay. On the receiving side, the Endpoint delivers the message to the destination OFT, which credits tokens to the recipient (via mint or unlock). The contract supports two operational modes configured at deployment:

- **MintBurn**: Burns tokens from the sender on outbound transfers and mints tokens to the recipient on inbound transfers. Minting is delegated to an external contract implementing the `Mintable` interface (in practice, the SAC Manager). This is the expected mode for USDT0 on Stellar, where the SAC Manager acts as the mint authority.
- **LockUnlock**: Locks tokens in the OFT contract on outbound transfers and releases them on inbound transfers, using standard SEP-41 `transfer` calls.

The SAC Manager contract serves as the **admin of the underlying Stellar Asset Contract (SAC)** for USDT0. Stellar's native asset model requires that admin operations (minting, clawback, authorization control) be performed by a designated admin address. The SAC Manager wraps these admin operations behind a role-based access control (RBAC) layer, ensuring that only authorized parties can invoke sensitive operations. Crucially, it implements the `Mintable` interface that the OFT calls during inbound cross-chain credit operations.

Contract Architecture

The OFT contract is structured as a core module with three pluggable extensions:

- **OFT Fee Extension** (`oft_fee.rs`): Implements a configurable fee mechanism charged on outbound transfers. Fees are expressed in basis points (BPS) and can be set globally (default) or per destination endpoint. Collected fees are transferred from the sender to a designated fee deposit address. When fees are active, dust (sub-unit rounding residuals from decimal conversion) is absorbed into the fee rather than remaining with the sender.
- **Pausable Extension** (`pausable.rs`): Provides an emergency pause mechanism that halts all send, receive, and quoting operations. Pausing and unpausing are gated by separate RBAC roles (`PAUSER_ROLE` and `UNPAUSER_ROLE`), allowing for operational separation of concerns.
- **Rate Limiter Extension** (`rate_limiter.rs`): Implements a leaky-bucket rate limiter that constrains transfer throughput per direction (inbound/outbound) and per remote endpoint. It supports two modes: **Net** mode (where opposite-direction releases reduce in-flight amounts, modeling net flow) and **Gross** mode (where releases are ignored, modeling absolute volume). Rate limits decay linearly over a configurable time window.

The SAC Manager is a standalone contract whose sole storage is a reference to the underlying SAC token address. It delegates all token operations to the SAC via `StellarAssetClient`, adding an RBAC gate on each operation.

Cross-Chain Message Flow

1. **Outbound (Send)**: The sender calls `send` on the OFT. The contract asserts the OFT is not paused, computes the fee (if any), calculates the debit amounts with decimal conversion and dust handling, debits tokens (burn or lock), consumes outbound rate limit capacity and releases inbound capacity, charges the fee to the fee deposit address, and submits the message to the LayerZero Endpoint.
2. **Inbound (Receive)**: The LayerZero Endpoint delivers a verified message via `lz_receive`. The OFT asserts it is not paused, credits tokens to the recipient (mint via SAC Manager or unlock), consumes inbound rate limit capacity and releases outbound capacity.

Decimal Conversion

The OFT uses a shared-decimals model to normalize token amounts across chains with different decimal precisions. A `decimal_conversion_rate` factor converts between local decimals (the native token precision on Stellar) and shared decimals (the common cross-chain precision). This conversion introduces "dust" -- amounts smaller than the conversion granularity -- which is handled differently depending on whether fees are active.

Security Model and Trust Assumptions

- **LayerZero Endpoint Trust:** The system fundamentally trusts the LayerZero V2 Endpoint and its configured security stack (DVNs, Executors) for message integrity and delivery. A compromised or misconfigured Endpoint could result in unauthorized minting/unlocking of tokens.
- **SAC Issuer Must Be Locked:** The SAC Manager's RBAC model is only effective if the Stellar classic issuer account has its master weight set to 0. If the issuer is not locked, it can mint tokens, authorize/deauthorize accounts, and clawback tokens directly via classic Stellar operations, completely bypassing the SAC Manager's access controls. This is an **external operational prerequisite** that cannot be enforced on-chain by the contracts themselves.
- **Upgradeability:** Both the OFT and SAC Manager contracts use `#[lz_contract(upgradeable(no_migration, rbac))]`, indicating they are upgradeable. An upgrade can replace the entire contract logic, making the upgrade authority a root-level trust assumption.
- **Delegate Authority:** At OFT construction, a `delegate` address is set as both the OApp delegate (for LayerZero Endpoint configuration) and the initial owner. This address can configure peers, message libraries, and other Endpoint-level settings that directly affect cross-chain security.
- **Rate Limiter as Soft Mitigation:** The rate limiter bounds cross-chain throughput but does not prevent exploits outright. It constrains the damage window in the event of a

compromised remote chain or Endpoint misconfiguration, buying time for operational response (e.g., pausing the contract).

- **Fee Deposit Address Requirement:** If a fee BPS is configured but no fee deposit address is set, outbound transfers will revert. This is by design -- there is no silent fee discard -- but it means an incomplete fee configuration can halt outbound operations.

Privileged Roles

The system employs granular role-based access control across both contracts. All role assignments are managed by the contract owner via the `RoleBasedAccessControl` trait.

OFT Contract Roles:

- **Owner** (via `Ownable`): Manages TTL, grants/revokes all RBAC roles, and controls upgrade authority. This is the root administrative role for the OFT contract.
- **AUTHORIZER:** Sets the fee deposit address via `set_fee_deposit_address`. This role controls where collected transfer fees are routed.
- **FEE_CONFIG_MANAGER_ROLE:** Configures default and per-destination fee rates in basis points via `set_default_fee_bps` and `set_fee_bps`.
- **PAUSER_ROLE:** Can pause the OFT, halting all send, receive, and quoting operations. This is an emergency control role.
- **UNPAUSER_ROLE:** Can unpause the OFT, resuming normal operations. Deliberately separated from the pauser role to allow independent operational control.
- **RATE_LIMITER_MANAGER_ROLE:** Configures rate limit parameters (limit, window, mode) per direction and per remote endpoint via `set_rate_limit`.

SAC Manager Contract Roles:

- **Owner** (via `Ownable`): Manages TTL, grants/revokes all RBAC roles, and controls upgrade authority for the SAC Manager.
- **ADMIN_MANAGER_ROLE:** Can change the SAC admin via `set_admin`. This is a highly sensitive operation -- transferring SAC admin to another address would remove the SAC Manager's control over the token entirely.
- **MINTER_ROLE:** Can mint tokens via the SAC. In the intended deployment, this role is granted to the OFT contract so it can mint USDT0 on inbound cross-chain credit operations.
- **BLACKLISTER_ROLE:** Can authorize or deauthorize accounts via `set_authorized`, enabling address-level blacklisting/whitelisting of token holders.

- **CLAWBACK_ROLE**: Can clawback (seize and burn) tokens from any account via `clawback`.

Low Severity

L-01 `quote_ofc` Can Return An Incorrect `max_amount_ld`

`quote_ofc` derives `max_amount_ld` by unconditionally overwriting the inherited limit with raw outbound rate-limit capacity:

```
limit.max_amount_ld = Self::rate_limit_capacity(env, &Direction::Outbound,
send_param.dst_eid);
```

This introduces two inconsistencies in the quoted maximum.

The first appears when no rate limit is configured. In that case, `rate_limit_capacity` falls back to `i128::MAX`, which replaces the bounded default established in `__quote_ofc`:

```
let limit = OFTLimit { min_amount_ld: 0, max_amount_ld: u64::MAX as i128 };
```

That replacement is unsafe at the quoting layer because actual sends are still bounded by the `u64` ceiling enforced in `to_sd`. As a result, `quote_ofc` may advertise a `max_amount_ld` that cannot be sent and would revert with `Overflow` during execution.

The second issue is a unit mismatch. Outbound rate-limit capacity is consumed using `amount_received_ld`, which is the post-fee amount:

```
Self::__consume_rate_limit_capacity(env, &Direction::Outbound, dst_eid,
amount_received_ld);
```

However, `max_amount_ld` is expected to represent the largest pre-fee input a user may pass to `send()`. Returning raw rate-limit capacity therefore underreports the true maximum whenever both fees and rate limits are enabled for the same destination. For example, with a 10% fee and 1,000,000 units of remaining capacity, the function reports 1,000,000 even though a user could send roughly 1,111,111 before fees and still remain within the configured post-fee limit.

Although this does not place funds at risk, it makes `quote_ofc` unreliable for integrators and frontends. Depending on configuration, the returned value can either be too large and cause a revert at execution time, or too small and unnecessarily restrict users.

Consider using the same bounded default value `u64::MAX as i128` in the event that no rate limit is configured. Additionally, consider applying the same fee back-calculation used in the console OFT implementation.

Update: Resolved in [pull request #213](#). The team stated:

The unbounded `i128::MAX` fallback is replaced with `u64::MAX as i128` as the bounded default. Fee back-calculation aligned with the console OFT implementation to ensure accurate `max_amount_id` reporting.

L-02 Fee Configuration Can Create an Unusable State When No Fee Deposit Address Is Set

The fee module allows privileged roles to configure a non-zero fee while the fee deposit address is unset. In that state, all user-facing flows that compute fees revert when they reach `__fee_view`, which requires `OFTFeeStorage::has_fee_deposit_address(env)` to be true.

This can happen in two ways. First, a `FEE_CONFIG_MANAGER_ROLE` account can set a default or destination-specific fee before an `AUTHORIZER` sets the fee deposit address. Second, an `AUTHORIZER` can clear the fee deposit address after fees have already been configured, because `__set_fee_deposit_address` accepts `None` without checking whether any non-zero fee remains active.

The result is a complete denial of service for transfers and quoting operations that pass through `__debit_view` and `__fee_view`, including `quote_of`, `quote_send`, and `send`. Although no funds are lost because execution reverts before token movement, the contract can be placed into a configuration where normal user operations are entirely unavailable until an administrator repairs the settings.

This is primarily a configuration-safety issue rather than an externally exploitable vulnerability, since only privileged roles can create the invalid state. Still, the current design permits an avoidable broken configuration and diverges from the console-OFT reference, which makes the fee deposit address mandatory and non-removable.

Consider adjusting configuration logic to enforce the invariant any non-zero fee requires a valid fee deposit address. When setting a default or per-destination fee to a non-zero value, the contract should require that a deposit address is already configured. Likewise, when clearing the fee deposit address, the contract should first ensure that no active non-zero default or destination-specific fee remains.

Update: Acknowledged, not resolved. The team stated:

Acknowledged. This scenario requires a privileged role to misconfigure the contract, and the resulting state produces clear, revert messages in affected operations rather than silent fund loss. While we agree that enforcing the invariant at the configuration layer is a reasonable improvement, we do not consider this a security vulnerability given the admin-only precondition and the existing error handling.

L-03 Pausing `__credit` Can Strand In-flight Cross-chain Transfers

The Soroban OFT pauses inbound settlement by calling

`Self::__assert_not_paused(env)` inside `__credit`. When a transfer has already been debited on the source chain and the destination chain is paused before delivery, the receive path reverts and the bridged tokens remain unavailable until an administrator unpauses and the LayerZero message is retried.

This is a meaningful deviation from the console-OFT reference, which explicitly avoids pausing the credit path to prevent in-flight token lockups, and it also differs from the EVM deployment pattern, where no equivalent OFT-level pause is enforced on credit. As implemented, a single global pause blocks settlement for messages arriving from every source chain, so any transfer already in flight at the moment of the pause becomes stuck without a user-controlled recovery path.

The condition does not usually cause permanent loss on its own because the message can normally be retried after unpause. However, it still creates an unbounded custody gap for users: tokens are already burned or locked on the source chain, but cannot be credited on the destination chain for as long as the pause remains active. Because the pause is global, the operational impact scales across all inbound routes simultaneously.

Consider removing the pause check from `__credit` to align the implementation with the reference behavior and avoid trapping in-flight transfers. If the intended policy is to halt inbound settlement during emergencies, consider documenting that behavior as an intentional deviation, and it consider separating outbound and inbound pause controls so already-debited transfers can still finalize.

Update: Acknowledged, not resolved. The team stated:

Acknowledged. When paused, in-flight transfers do not result in permanent fund loss – messages remain retryable and tokens are not permanently locked. We consider this an

acceptable operational behavior given the bounded unpause window. May document this as operational guidance for administrators.

L-04 Blacklisted Recipients Can Cause Stuck Cross-Chain Messages

In the MintBurn flow, inbound messages ultimately credit tokens by calling `MintableClient::mint(to, amount, ...)`. When the underlying token is a Stellar SAC managed through the SAC Manager, this path reaches SAC `mint`, which enforces the recipient's authorization status. If the destination address has been deauthorized through `set_authorized(id, false)`, the mint reverts with a deauthorization error instead of crediting the transfer.

Because message clearing and `lz_receive` execution occur in the same transaction, that revert causes the entire receive flow to roll back. At that point, the source-chain burn has already occurred, but the destination-chain credit is not completed, leaving the message stuck in a retryable state at the endpoint. The funds are therefore not permanently lost, but they are unavailable until an authorized operator re-enables the recipient and the message is executed again.

This is not caused by a flaw in the OFT's minting logic itself, but by an integration assumption that does not hold for Stellar SAC tokens: even admin-driven mint operations still require the recipient to be authorized. In a cross-chain setting, that behavior creates a temporary fund-lock condition with no self-service recovery path for the user. The issue is also easy to miss in testing because a permissive mock token can mask the authorization check performed by the real SAC implementation.

Consider documenting this behavior explicitly as an operational limitation of bridging to blacklisted recipients, and consider including deauthorized destinations in integration tests.

Update: *Acknowledged, not resolved. The team stated:*

Acknowledged. This is a known Stellar SAC integration constraint, not an OFT logic flaw — funds are not permanently lost as the message remains retryable. We will document this as an operational limitation in the integration guide.

Notes & Additional Information

N-01 LockUnlock Mode Lacks End-to-End Test Coverage

The end-to-end test suite does not exercise `OftType::LockUnlock`. The shared integration setup hardcodes `OftType::MintBurn`, so all existing end-to-end coverage validates only the `MintBurn` path. As a result, the distinct `LockUnlock` debit and authorization flow is never validated in the same environment as fees, rate limits, and pausability.

This gap is meaningful because `LockUnlock` does not share the same debit behavior as `MintBurn`. In particular, the current test helpers are built around `burn` sub-invocations, while `LockUnlock` performs debit operations through `transfer`. The existing infrastructure therefore cannot be reused to validate the `LockUnlock` flow without modification. Although isolated unit tests exist for `LockUnlock`, they rely on a harness that directly implements `OFTInternal`, excludes extensions, and uses permissive mocked authorization, which does not provide confidence that the full composed path behaves correctly in production-like conditions.

No concrete vulnerability was identified during review. The issue is that a separate operating mode, with distinct auth expectations, is effectively untested end-to-end. This reduces assurance that integrations between `LockUnlock` and surrounding features will continue to behave correctly as the codebase evolves.

Consider adding a dedicated end-to-end setup for `LockUnlock`, together with helper utilities that model `transfer`-based authorization trees rather than `burn`-based ones. The full send, receive, and quote flow should then be exercised under `LockUnlock`, including combinations with fees, rate limiting, and pausability.

Update: Resolved in [pull request #213](#). The team stated:

A dedicated LockUnlock E2E setup has been added, covering the full send, receive, and quote flow — including combinations with fees, rate limiting, and pausability.

N-02 Unused `SACAdminWrapperClient`

The `#[contract_trait]` macro on `SACAdminWrapper` generates `SACAdminWrapperClient`, which is re-exported from `sac-manager/src/lib.rs` but never used by the `oft/` crate. The OFT only interacts with the SAC Manager through `MintableClient`, and only invokes `mint`, so this extra export currently serves no purpose.

Consider removing the re-export.

Update: *Acknowledged, not resolved. The team stated:*

Acknowledged. The re-export is intentional — it enables downstream contracts (e.g., multisig) to interact with the SAC Manager directly.

N-03 `unwrap()` Breaks Error Pattern

Two storage reads use plain `.unwrap()`:

- `OFTStorage::oft_type(env).unwrap()`
- `SACManagerStorage::sac_token(env).unwrap()`

This is inconsistent with the rest of the codebase, which uses typed errors through `assert_with_error!` or `unwrap_or_panic` with explicit error enums. If either storage slot is unset, these calls will trigger a generic Soroban panic instead of a descriptive contract error, making failures harder to diagnose and less consistent for integrators.

Consider replacing these calls with the existing typed-error pattern used elsewhere in the codebase.

Update: *Acknowledged, not resolved. The team stated:*

Acknowledged. Both storage values are set at construction with no code path to unset them, making `.unwrap()` safe in practice.

Conclusion

The USDT0 Stellar/Soroban contracts implement LayerZero's OFT standard on the Stellar blockchain, enabling cross-chain USDT0 transfers through a mint-and-burn architecture backed by the SAC Manager as mint authority. The implementation follows the LayerZero console-OFT reference design, with extensions for fee collection, pausability, and rate limiting, and a separate SAC Manager contract providing role-gated admin operations over the underlying Stellar Asset Contract.

The audit identified low-severity and informational findings. The low-severity findings relate to operational edge cases in pause behavior, blacklist interactions with cross-chain messaging, fee configuration sequencing, and quoting accuracy. The informational findings cover code hygiene items, and test coverage observations. No critical- or high-severity vulnerabilities were found.

Overall, the codebase is well-structured and makes appropriate use of Soroban's contract patterns and RBAC model. The Everdawn team is appreciated for being highly responsive and providing valuable support throughout the audit process.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.